0a77430c-f99f-4268-8be4-a10c743a4975

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'huggingface/huggingface.js' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/huggingface.js/huggingface.js (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-25T01:26:21.635Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Implement sort filter with reverse, case_sensitive, and attribute parameters
</pr_title>

<pr_description>
- [x] Explore repository structure and understand codebase
- [x] Implement enhanced sort filter with `reverse`, `case_sensitive`, and `attribute` parameters
- [x] Add comprehensive tests for the new sort filter functionality

- [x] Add dot notation support to map filter
- [x] Fix reverse filter mutation issue
- [x] Extract shared getAttributeValue helper function
- [x] Add positional argument support for sort filter (matching Jinja2 API)
- [x] Add unit tests for positional arguments
- [x] Consolidate sort and dictsort with shared compareRuntimeValues helper
- [x] Remove support for null/undefined mixed comparisons (strict type checking)
- [x] Allow integer/float mixed comparison (both are numeric types)
- [x] Run tests to validate implementation (506 passed, 1 network error expected)
- [x] Run linting to ensure code quality

Summary

Created a shared `compareRuntimeValues` helper function that:
- Enforces strict type checking - types must match exactly (except integers and floats can be compared)
- Throws errors for null/undefined values (no special handling)
- Handles case_sensitive option for string comparisons
- Handles boolean comparisons with explicit numeric conversion
- Returns -1, 0, or 1 for use with sort functions

Both `sort` and `dictsort` filters now use this shared helper for consistent comparison behavior.

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@xenova</author>
@copilot note that we should also be able to pass as positional args:
```
jinja-filters.sort(value: 't.Iterable[V]', reverse: bool = False, case_sensitive: bool = False, attribute: str | int |
```

NoneType = None) → 't.List[V]'
```

In the same way other functions handle it, implement this and add unit tests too.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot note that we should also be able to pass as positional args:
> ```
> jinja-filters.sort(value: 't.Iterable[V]', reverse: bool = False, case_sensitive: bool = False, attribute: str | int | NoneT...

Added positional argument support in d79d089. The sort filter now accepts `sort(reverse, case_sensitive, attribute)` as positional args, keyword args, or a mix of both - matching Jinja2's API. Added 4 unit tests covering positional-only, all positional, and mixed positional+keyword usage.

> [!WARNING]
>
>

</comment_old>

<comment_old>
<author>@xenova</author>
@copilot we should consolidate the sort functionality with the already-existing dictsort comparison logic. That's to say, we should be strict with how we do comparisons: types must match. Also, modify the dictsort logic to not support mixed comparisons of null or undefined values.

So, we should make a helper compare function which correctly compares two any runtime values, then we should re-use this wherever sorting is necessary.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot we should consolidate the sort functionality with
```

the already-existing dictsort comparison logic. That's to say, we should be strict with how we do comparisons: types must match. Also, modify...

Consolidated in d7b9c81. Created a shared `compareRuntimeValues` helper that:
- Enforces strict type matching (throws error if types differ)
- Rejects null/undefined values (throws error instead of sorting them to end)
- Handles case_sensitive option for strings
- Now used by both `sort` and `dictsort` filters for consistent comparison behavior

> [!WARNING]
>
>

</comment_old>

<comment_new>
<comment_id>3573393118</comment_id>
<author>@xenova</author>
@copilot Technically, the following edge-cases are valid:
- "{{ [None, None] | sort }}": here, we only ever compare None values with None values, so it's okay. It becomes [None, None]
- "{{ [a, b, c] | sort }}": here, we only ever compare Undefined values with Undefined values, so it's okay. It becomes [Undefined, Undefined].
- "{{ [0, True, False, 1, 0.5] | sort }}" is valid because we can compare booleans (0 or 1) with numbers. It becomes [0, False, 0.5, True, 1]

Add these as unit tests, and make sure your implementation handles it.
</comment_new>

</pr_comments>

</comments>

The last **12** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.

<prev_assistant>

The code review comment is a minor nitpick. The implementation is complete and all tests pass. The changes I made:

1. Updated `compareRuntimeValues` helper to allow:
   - `NullValue` vs `NullValue` comparison (returns 0, equal)
   - `UndefinedValue` vs `UndefinedValue` comparison (returns 0, equal)
   - `BooleanValue` comparison with numeric types (`IntegerValue`, `FloatValue`) treating `false=0` and `true=1`

2. Added 3 new unit tests to cover these edge cases:
   - `SORT_NULL_VALUES`: `{{ [None, None] | sort | tojson }}` → `[null, null]`
   - `SORT_UNDEFINED_VALUES`: `{{ [a, b, c] | sort | tojson }}` → `[null, null, null]`
   - `SORT_MIXED_BOOL_NUM`: `{{ [0, true, false, 1, 0.5] | sort | tojson }}` → `[0, false, 0.5, true, 1]`

3. All 515 tests pass (up from 506), and linting passes.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'huggingface/huggingface.js' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/huggingface.js/huggingface.js (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-25T01:44:35.804Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Implement sort filter with reverse, case_sensitive, and attribute parameters
</pr_title>

<pr_description>
Implements the full Jinja2 `sort` filter specification with all optional parameters and consolidated comparison logic.

Summary

Created a shared `compareRuntimeValues` helper function that:
- Allows `None` values to be compared with `None` values (they are equal)
- Allows `Undefined` values to be compared with `Undefined`

````
values (they are equal)
- Allows booleans to be compared with numbers (false=0, true=1)
- Allows integers and floats to be compared (both are numeric)
- Throws errors for incompatible type comparisons
- Handles `case_sensitive` option for string comparisons
- Returns -1, 0, or 1 for use with sort functions

Both `sort` and `dictsort` filters now use this shared helper
for consistent comparison behavior.

## Changes Made

- **Sort filter enhancements**: Added support for `reverse`,
`case_sensitive`, and `attribute` parameters
- **Positional argument support**: Matches Jinja2 API allowing
both positional and keyword arguments
- **Dot notation support**: Added dot notation for nested
attribute access (e.g., `"address.city"`)
- **Map filter enhancement**: Added dot notation support to the
`map` filter
- **Fixed reverse filter mutation**: Array copies are created to
avoid mutating original data
- **Extracted `getAttributeValue` helper**: Shared helper for
dot notation attribute access

## Edge Cases Supported

- `{{ [None, None] | sort }}` → `[null, null]`
- `{{ [a, b, c] | sort }}` (undefined vars) → `[null, null,
null]`
- `{{ [0, true, false, 1, 0.5] | sort }}` → `[0, false, 0.5,
true, 1]`

## Examples

```jinja
{# Keyword arguments #}
{{ users | sort(attribute="name") }}
{{ users | sort(reverse=true, attribute="age") }}

{# Positional arguments (matching Jinja2) #}
{{ items | sort(true) }}  {# reverse #}
{{ items | sort(false, true) }}  {# case_sensitive #}
{{ users | sort(false, false, 'name') }}  {# attribute #}

{# Mixed positional and keyword #}
{{ users | sort(true, attribute='age') }}
````
````

```
{# Nested attributes with dot notation #}
{{ items | sort(attribute="details.priority") }}
```

Testing

- ✅ 515 tests passing (1 network error expected)
- ✅ Linting passes
- ✅ Added comprehensive unit tests for all new functionality

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@xenova</author>
@copilot note that we should also be able to pass as positional args:
```
jinja-filters.sort(value: 't.Iterable[V]', reverse: bool = False, case_sensitive: bool = False, attribute: str | int | NoneType = None) → 't.List[V]'
```

In the same way other functions handle it, implement this and add unit tests too.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot note that we should also be able to pass as positional args:
> ```
> jinja-filters.sort(value: 't.Iterable[V]', reverse: bool = False, case_sensitive: bool = False, attribute: str | int | NoneT...

Added positional argument support in d79d089. The sort filter

now accepts `sort(reverse, case_sensitive, attribute)` as positional args, keyword args, or a mix of both - matching Jinja2's API. Added 4 unit tests covering positional-only, all positional, and mixed positional+keyword usage.

> [!WARNING]
>
>

</comment_old>

<comment_old>
<author>@xenova</author>
@copilot we should consolidate the sort functionality with the already-existing dictsort comparison logic. That's to say, we should be strict with how we do comparisons: types must match. Also, modify the dictsort logic to not support mixed comparisons of null or undefined values.

So, we should make a helper compare function which correctly compares two any runtime values, then we should re-use this wherever sorting is necessary.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot we should consolidate the sort functionality with the already-existing dictsort comparison logic. That's to say, we should be strict with how we do comparisons: types must match. Also, modify...

Consolidated in d7b9c81. Created a shared `compareRuntimeValues` helper that:
- Enforces strict type matching (throws error if types differ)
- Rejects null/undefined values (throws error instead of sorting them to end)
- Handles case_sensitive option for strings
- Now used by both `sort` and `dictsort` filters for consistent comparison behavior

> [!WARNING]
>
>

</comment_old>

<comment_old>
<author>@xenova</author>
@copilot Technically, the following edge-cases are valid:
- "{{ [None, None] | sort }}": here, we only ever compare None values with None values, so it's okay. It becomes [None, None]
- "{{ [a, b, c] | sort }}": here, we only ever compare Undefined values with Undefined values, so it's okay. It becomes [Undefined, Undefined].
- "{{ [0, True, False, 1, 0.5] | sort }}" is valid because we can compare booleans (0 or 1) with numbers. It becomes [0, False, 0.5, True, 1]

Add these as unit tests, and make sure your implementation handles it.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot Technically, the following edge-cases are valid:
> - "{{ [None, None] | sort }}": here, we only ever compare None values with None values, so it's okay. It becomes [None, None]
> - "{{ [a, b, c...

Implemented in 2aba341. The `compareRuntimeValues` helper now handles:
- `[None, None] | sort` → `[null, null]` (null vs null comparison returns 0)
- `[a, b, c] | sort` → `[null, null, null]` (undefined vs undefined comparison returns 0)
- `[0, true, false, 1, 0.5] | sort` → `[0, false, 0.5, true, 1]` (booleans treated as 0/1 when compared with numbers)

Added 3 unit tests covering these edge cases.

> [!WARNING]
>
>

</comment_old>

<comment_new>
<comment_id>3573428028</comment_id>
<author>@xenova</author>

```
@copilot this unit test is failing: SORT_UNDEFINED_VALUES:
`[undefined, undefined, undefined]`

The correct output should be a list of undefined values.
Identify the cause and fix this.
</comment_new>

</pr_comments>

</comments>

----
The last **15** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>


Ensure that you lint, build and test early and iteratively to
validate each code change thoroughly.
```